

Blynk: A Privacy-First, Browser-Based Eye Strain and Posture Monitor

Project Proposal for UCS503P

Submitted to : Asif

Thapar Institute of Engineering and Technology

Hardik Chandna, CSED

August 2026

1. Higher-order goal : Secondary framing

This project aims to make digital wellbeing something people can actually see and act on, rather than something they vaguely worry about. Long screen sessions quietly cause eye strain and poor posture, but most people have no real feedback on how bad it is getting until it already hurts. The immediate engineering objective is to build a browser-based tool that measures this in real time and nudges the user toward genuine recovery, not just a reminder that gets dismissed.

2. Time-to-value : Primary heuristic

- We will deliver a working core loop early – camera in, live blink and distance readings out – and layer the dashboard, break-enforcement feature, and history tracking on top of it incrementally.
- We will prioritize a functioning end-to-end pipeline over a polished but incomplete one, since a browser-based, no-backend architecture means every layer we add should keep working without needing external infrastructure.
- Success in the first iteration is defined narrowly: a page that reliably detects blinks and estimates screen distance from a live webcam feed, with visible output. Everything else builds on top of that.

3. Problem Statement

Students and professionals routinely spend six to ten hours a day on screens, often in long, uninterrupted stretches. This has two well-documented physical costs:

- **Reduced blink rate:** when people focus on a screen, they blink significantly less than normal, which dries out and strains the eyes over time – a pattern often referred to as digital eye strain or Computer Vision Syndrome.
- **Poor posture:** long sessions hunched toward a screen contribute to chronic neck and shoulder strain.

The tools that exist to address this are largely passive. Break-reminder apps fire a timer every twenty minutes but have no way of knowing whether the user actually rested their eyes or simply dismissed the notification

and kept working. Meanwhile, most camera-based wellness tools that *do* try to measure something meaningful require uploading video to a server, which raises real privacy concerns for something this personal.

Why this matters now: screen time has only gone up, and the gap between “being reminded to take a break” and “actually taking one” is where most of these tools fail. A system that can measure strain directly and verify recovery, without ever sending a user’s camera feed anywhere, addresses a real and current gap.

4. Proposed Solution : Software Proposition

4.1. Overview

Build a browser-based wellness monitor, Blynk, with the following components:

- **Live tracking:** on-device computer vision tracks facial landmarks from the webcam feed to calculate blink rate and estimated screen distance.
- **Real-time dashboard:** gauges for blink rate and screen distance, plus an ambient visual that shifts as strain builds up, giving the user a sense of their state without needing to read numbers directly.
- **Enforced recovery:** following the 20-20-20 rule, the app triggers a break when sustained strain is detected, and verifies – using the same live tracking – that the user actually followed a moving on-screen target with their eyes, rather than simply waiting out a timer.
- **History tracking:** daily and weekly trends stored locally, so the user can see their own patterns over time.

4.2. Core workflow

1. User opens the app and grants camera access; a short one-time calibration step records their baseline screen distance.
2. The app continuously analyzes the live video feed on-device to compute blink rate and relative screen distance.
3. The dashboard updates in real time, and the ambient visual reflects the user’s current strain level.
4. When sustained strain is detected, the app triggers a verified eye-exercise break; the screen unlocks only once the exercise is genuinely completed.
5. Metrics are logged locally so the user can review trends on a history page.

4.3. Operational constraints

- No video frame is ever transmitted or stored – all inference happens client-side, in the browser.
- The system should run smoothly on a typical laptop webcam and browser, without requiring specialized hardware.
- Avoid dependence on training custom models; use established, pretrained landmark-detection models and focus engineering effort on the analysis logic and user-facing system built on top of them.

5. Solution Approach : Engineering focus

This is an engineering course project, so the focus is on building a reliable real-time pipeline and a genuinely usable interface, not on inventing new computer vision research.

5.1. Detection logic : non-research

Blink and distance detection will use an established, pretrained facial-landmark model (MediaPipe Face Landmarker) rather than a custom-trained one:

- Blinks are detected using the eye-aspect-ratio technique on eyelid landmark points, with a threshold tuned against real test data collected on the author's own face across varying lighting conditions.
- Screen distance is estimated from interpupillary pixel distance, calibrated once per session against a known reference distance.
- Thresholds and calibration are treated as tunable parameters, not fixed constants, since face geometry and camera setups vary between users.

5.2. Web architecture

A fully client-side architecture:

- **Frontend**: React for the dashboard and interface, Framer Motion for animated feedback, Three.js (via react-three-fiber) for the ambient visual, and D3.js for history charts.
- **Inference layer**: MediaPipe's face landmark model, running in-browser via WebAssembly, processing the live webcam feed frame by frame.
- **Storage**: IndexedDB, the browser's built-in local database, used to persist daily metrics for the history view – no external database or server involved.

5.3. Iterative delivery

Development will proceed in the phased order described in Section 9, with each phase producing something demonstrable before the next begins, rather than building all layers in parallel.

6. Evaluation Criterion : measurable and attributable

6.1. Primary evaluation metric

Blink-detection reliability: agreement between the app's detected blink count and a manually counted ground truth over a fixed test session.

- Target: within a reasonable margin of manual count under normal lighting, established through direct testing on the author.
- Attribution: measured by comparing logged blink events against a manually annotated recording of the same session.

6.2. Secondary evaluation metrics

- **Distance-estimation consistency:** the app’s relative “too close” signal should trigger consistently at the same physical distance across repeated tests, following calibration.
- **Break-verification accuracy:** whether the enforced eye-exercise feature correctly distinguishes genuine target-following from a user simply waiting out the timer.
- **Real-time performance:** frame processing should sustain a usable rate on a standard laptop without noticeably degrading browser responsiveness.
- **Data privacy:** verified directly by inspecting the browser’s network activity during a live session to confirm no video or frame data is ever transmitted.

6.3. Validation plan

Given the personal, single-user nature of this tool, validation will primarily be self-testing across varied conditions – different lighting, with and without glasses, different times of day – supplemented by a small number of informal test sessions with classmates to check that the calibration step generalizes beyond the author’s own face.

7. Scalability : with foresight

1. **Scalable (theoretically):** the architecture separates the sensing/inference layer from the presentation layer – the dashboard, ambient visual, and history charts all simply consume a common metrics data shape. This means additional visualizations or a second tracking model (for posture, for instance) can be added without restructuring existing components.
2. **Operationally deployable (practically):** since the entire application runs client-side with no backend, it can be deployed as a static site on any standard web host (e.g., Vercel or Netlify) with no server provisioning or database management required.

8. Engine availability heuristic

A mature set of tools is available to implement this without inventing new infrastructure:

- Pretrained facial-landmark detection (MediaPipe Face Landmarker).
- Browser-native ML runtime (WebAssembly / TensorFlow.js).
- Standard browser camera access (getUserMedia).
- Browser-native local storage (IndexedDB).
- Established frontend libraries for interface, animation, 3D visuals, and charting (React, Framer Motion, Three.js, D3.js).

These mature components let the project focus on the analysis logic, calibration, and the verified-recovery interaction, rather than on building tracking infrastructure from scratch.

9. Project scope and deliverables : iteration-friendly

9.1. Initial deliverable : first iteration

- Live webcam feed with working blink-rate and screen-distance detection.
- Real-time dashboard with animated gauges.
- Ambient visual reacting to live strain metrics.
- Local history logging (IndexedDB) with basic trend charts.

9.2. Subsequent deliverables

- Verified eye-exercise break feature (camera-checked target following).
- Refined calibration flow for screen distance.
- *Stretch*: posture detection using a second pretrained landmark model on the same pipeline.
- *Stretch*: a lightweight conversational assistant that summarizes a user’s stored weekly data on request, backed by a minimal single-endpoint server.
- *Stretch*: a native desktop widget (via Electron) reusing the same tracking and dashboard components.

10. Risks and mitigations

- **Detection reliability in poor conditions**: low light, glasses glare, or side angles can degrade landmark detection. Mitigation: test extensively under varied conditions early, and treat thresholds as tunable rather than fixed.
- **Performance on lower-end devices**: running continuous inference in-browser can be resource-intensive. Mitigation: throttle frame-processing rate rather than analyzing every frame, since blink and distance patterns do not require 60 fps sampling.
- **Calibration generalizing beyond one user**: a calibration step tuned only on the author’s own face may not generalize well. Mitigation: keep the calibration step user-driven and repeatable rather than relying on fixed defaults.
- **Scope creep**: the project has several natural stretch directions (posture, chat assistant, desktop widget). Mitigation: these are explicitly scoped as post-core deliverables, attempted only once the primary pipeline is fully working.

11. Summary

This project proposes Blynk, a deployable, browser-based wellness tool that addresses a real and growing problem – unmanaged eye strain and poor posture during long screen sessions – using on-device computer vision rather than a server-dependent approach. It meets the course criteria by emphasizing an incrementally deliverable core pipeline, a system architecture built entirely on mature, available tooling rather than novel research, and evaluation criteria that are directly measurable through self-testing and live inspection of the app’s network behavior. The project remains scoped tightly around a working core loop, with clearly separated stretch goals for posture tracking, an AI-assisted history summary, and a desktop widget.